

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPICS 1.19 - 1.21

Outliers, Matplotlib & Seaborn

Day 20 — finding the strange values, and seeing the data

3

Topics in one session

90 min

Concepts + live code

30 min

Practice notebook

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Three topics, one idea

They are the same job

An outlier is a number that looks wrong. You FIND it with arithmetic — the IQR rule, a z-score — but you can only DECIDE about it by looking at the data. And looking at data means plotting it.

So: detect, then draw, then judge.

Four questions from Day 19

Does `isna()` find -999?

→ No — use `na_values`.

`fillna(mean)` does what to std?

→ Shrinks it.

Normalise or dedupe first?

→ Normalise. Always.

Why ask WHY a value is missing?

→ The gaps may not be random.

A word on today's pace

Three topics in one session means we cover the essentials of each rather than everything. That is deliberate: plotting is a skill you build by doing, not by being shown every argument. You will get the core twenty per cent that handles eighty per cent of real work, and tomorrow's full EDA project is where it becomes fluent. The practice notebook is longer than usual for the same reason.

How today's two hours are structured

1

Outliers

IQR, z-scores, and what to do about them

30 min

2

Matplotlib

fig, ax — and the four core plots

30 min

3

Seaborn

Statistical plots straight from a DataFrame

25 min

4

Mini build

A one-screen look at a new dataset

5 min

5

Practice notebook

You code, I walk around

30 min

Tomorrow — Day 21

A full EDA on a real dataset: everything from Days 15 to 20, start to finish.



Outliers

Error, or the finding? · 30 minutes

- The IQR rule
- Z-scores and their blind spot
- Four treatments

The same value, three different answers

1 A typo

A height of 1700 cm

FIX or remove it — it is not real

2 A sentinel

An age of -999

Day 19: it means missing

3 A real extreme

One customer spending 50x the rest

KEEP it — it may be the finding

No test can tell you which of the three you have

The IQR rule and the z-score both answer one question only: “is this value far from the others?” They cannot answer “is it wrong?” — that needs you to know what the data MEANS. A fraud detection system exists entirely to find outliers; deleting them would delete the product. The techniques below find candidates. You decide.

Two detectors — and why one of them fails here

The 1.5 x IQR rule — Day 14's bounds

```
d = [12, 15, 14, 13, 16, 15, 14, 95, 13]

q1, q3 = d.quantile([.25, .75])    13.0  15.0
iqr    = q3 - q1                    2.0
lower  = q1 - 1.5 * iqr             10.0
upper  = q3 + 1.5 * iqr             18.0

d[(d < lower) | (d > upper)]      -> [95]
```

Quartiles ignore the extremes, so the outlier cannot move its own bounds.

The z-score misses it completely

```
z = (d - d.mean()) / d.std()

mean 23.0    std 27.03
max |z| -> 2.66

# the usual threshold is 3
# so 95 is NOT flagged
```

The 95 inflated the mean AND the std, so it hid behind its own distortion.

Why this happens, and what it means for your choice

Against the clean data (mean 14.0, std 1.31) the 95 has a z-score of 61.9. Against the contaminated data it scores 2.66. The z-score is built from the mean and the standard deviation, and Day 14 told you both are dragged by extremes — so the very value you are hunting corrupts the detector looking for it. Use the IQR rule by default; use z-scores only when you already know the data is roughly normal and free of gross errors.

Four things you can do — and what each costs

Option	Code	Cost
Keep it	— nothing	every mean and std stays distorted
Remove the row	<code>df[df.v <= upper]</code>	loses a real observation; biases if not random
Cap it (winsorise)	<code>df.v.clip(lower, upper)</code>	keeps the row, invents a value
Transform	<code>np.log1p(df.v)</code>	compresses extremes; changes the units

Capping, with Day 13's clip()

```
df["v"] = df["v"].clip(lower, upper)
# 95 becomes 18.0 — the row survives
```

Often the best compromise: you keep the customer in the dataset but stop one value dominating every statistic. Say in your write-up that you capped, and at what.

The rule for all four

Never treat an outlier you have not looked at. Print the actual rows first — the value, and everything else about that record. Half the time the context tells you immediately whether it is a typo, a sentinel or a genuinely remarkable customer, and the decision makes itself.

```
df[df["v"] > upper] # LOOK
```

2

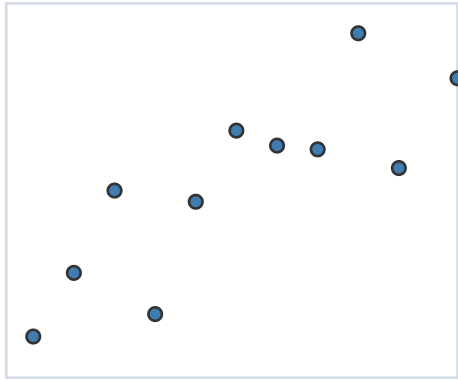
Matplotlib

The four plots you actually need · 30 minutes

- Why plot at all
- fig, ax
- line · bar · scatter · hist
- Labels & saving

Four datasets. Identical statistics.

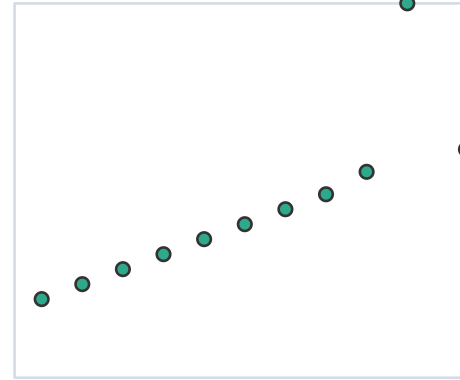
dataset I



dataset II



dataset III



dataset IV



All four have the same mean, the same standard deviation, and the same correlation

```
mean x  9.00    mean y  7.50
std  y  2.03    r      0.816
# for ALL FOUR, to three decimals
```

This is Anscombe's quartet, built in 1973 to make exactly this point. One is roughly linear, one is a curve, one is a line with a single outlier, and one is a vertical stack with a single distant point. Every summary statistic you learned on Day 14 is blind to the difference. Plotting takes ten seconds and is not optional.

fig, ax — the one habit to build

The style you will see online

```
plt.plot(x, y)
plt.title("Sales")
plt.xlabel("Month")
plt.show()
```

Works for one quick chart. It draws on whatever figure happens to be current, so the moment you want two plots side by side it falls apart.

The style to learn

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(x, y)
ax.set_title("Sales")
ax.set_xlabel("Month")
```

fig is the canvas, ax is the one set of axes you are drawing on. Named, so there is never any doubt which plot you are changing.

Two plots side by side — the reason this matters

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(df["age"])
axes[1].scatter(df["age"], df["salary"])
fig.tight_layout()
```

axes is a NumPy array of Axes objects — Day 12's indexing, applied to charts. For a grid use `plt.subplots(2, 2)` and index with `axes[0, 1]`. And always finish with `fig.tight_layout()`, which stops the labels of one plot overlapping the next.

The four plots that cover most of your work

line

a value over time

```
ax.plot(x, y)
```

only when x has a real order

bar

comparing categories

```
ax.bar(names, vals)
```

sort first, or it reads as noise

scatter

two numbers, related?

```
ax.scatter(x, y)
```

add alpha=0.5 when points overlap

hist

the shape of one column

```
ax.hist(v, bins=20)
```

bins changes the story — try a few

A chart without labels is not finished

```
ax.set_xlabel("Month")    ax.set_ylabel("Sales (Rs)")
ax.set_title("Monthly sales, 2026")
ax.legend()               fig.savefig("out.png", dpi=150,
                           bbox_inches="tight")
```

Units in the axis label, a title that says what the reader is looking at, and a legend whenever there is more than one series. `bbox_inches="tight"` stops `savefig` cropping your labels off, which it will otherwise do.

3

Seaborn

Statistical plots from a DataFrame · 25 minutes

- Why seaborn
- Five plots
- hue
- The barplot trap

Seaborn — matplotlib that understands DataFrames

Matplotlib: you do the grouping

```
for city in df["city"].unique():
    sub = df[df["city"] == city]
    ax.scatter(sub["age"], sub["spend"],
               label=city)
ax.legend()
```

Seaborn: you name the column

```
sns.scatterplot(data=df,
                x="age",
                y="spend",
                hue="city")
```

Same chart, legend included. hue= splits by any column.

It is matplotlib underneath — which is why both matter

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.boxplot(data=df, x="city", y="spend", ax=ax)
ax.set_title("Spend by city")      # matplotlib again
```

Every seaborn function takes `ax=` and returns an `Axes`, so you draw with seaborn and then adjust with matplotlib. Learn seaborn for the statistical defaults and matplotlib for the control — they are one toolkit, not two competing ones.

Five seaborn plots that answer real questions

Question	Plot	Why it
What shape is this column?	<code>sns.histplot(data=df, x="age")</code>	distribution, skew, gaps
Does this group differ?	<code>sns.boxplot(data=df, x="city", y="v")</code>	median, spread, OUTLIERS as dots
Are these two related?	<code>sns.scatterplot(..., hue="city")</code>	relationship, split by a category
What correlates with what?	<code>sns.heatmap(df.corr(numeric_only=True))</code>	the whole correlation matrix at once
How many of each?	<code>sns.countplot(data=df, x="city")</code>	Day 15's value_counts, drawn

The boxplot is the outlier plot

Its whiskers are drawn at exactly the 1.5 x IQR bounds from section 1, and anything beyond them is plotted as an individual dot. So a boxplot is section 1 and section 3 in one picture — the detection rule, drawn.

One more worth knowing

```
sns.pairplot(df, hue="city")
```

Every numeric column against every other, in one grid. Slow on wide data — use it on five or six columns, not fifty.

The seaborn trap — barplot shows the MEAN

! Not the total

```
df   g   v
    A   10
    A   20
    B   30

sns.barplot(data=df, x="g", y="v")
-> A shows 15      B shows 30
# A is the MEAN of 10 and 20, not the sum
```

Both bars look equal-ish, and a reader assumes they are totals.

Say what you mean

```
sns.barplot(data=df, x="g", y="v",
            estimator="sum")

# or group it yourself first:
t = df.groupby("g", as_index=False)["v"].sum()
sns.barplot(data=t, x="g", y="v")
```

The second is clearer — you can see the numbers before they are drawn.

And those little lines on top of the bars

Seaborn draws a confidence interval on every bar by default — the thin vertical line through the top. It is genuinely useful information, but only if you know it is there and what it means. If you are plotting totals rather than estimates it is meaningless, so turn it off with `errorbar=None`. The broader lesson: before you show anyone a seaborn chart, check what the default actually computed.

Choosing the right chart

You want to show	Use	Avoid
One numeric column's shape	<code>histplot</code> or <code>boxplot</code>	a bar of the mean — it hides the shape
A number per category	<code>barplot</code>	pie charts; and sort the bars first
Change over time	<code>lineplot</code>	bars, unless the gaps matter
Two numbers together	<code>scatterplot</code>	two separate line charts
Many columns at once	<code>heatmap of corr()</code>	a wall of scatter plots
Counts of a category	<code>countplot</code>	a table, if there are more than five

The honest test: show the chart to someone who has not seen the data and ask them what it says. If they get it wrong, the chart is wrong — not the reader. That test will improve your plots faster than any list of rules, and it is the standard your Day 21 project will be marked against.

Debug corner — eight plotting and outlier bugs

You wrote	What happens	Fix
<code>z-score on dirty data</code>	the outlier hides behind its own effect	use the IQR rule
<code>deleting every outlier</code>	you may delete the finding	LOOK at the rows first
<code>plt.plot()</code> for two charts	both land on the same axes	<code>fig, ax = plt.subplots()</code>
<code>no axis labels</code>	nobody can read it, including you	<code>ax.set_xlabel(...)</code>
<code>sns.barplot</code> for totals	it shows the MEAN	<code>estimator="sum"</code>
<code>df.corr()</code> with text columns	ValueError	<code>corr(numeric_only=True)</code>
<code>savefig</code> cutting off labels	the default crops tightly	<code>bbox_inches="tight"</code>
<code>overlapping subplot labels</code>	unreadable grid	<code>fig.tight_layout()</code>

Rows 1 and 2 cost you correctness. Rows 3 to 8 cost you marks. All eight are in today's practice notebook.

Putting it together — one screen, one new dataset

```
import matplotlib.pyplot as plt
import seaborn as sns

# ---- 1. find the outliers, do not act yet
q1, q3 = df["spend"].quantile([.25, .75])
iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr

odd = df[(df["spend"] < lo) | (df["spend"] > hi)]
print(f"{len(odd)} outliers of {len(df)} rows")
print(odd) # LOOK at them

# ---- 2. four views, one figure
fig, axes = plt.subplots(2, 2, figsize=(11, 7))

sns.histplot(data=df, x="spend", bins=20, ax=axes[0, 0])
axes[0, 0].set_title("Spend: distribution")

sns.boxplot(data=df, x="city", y="spend", ax=axes[0, 1])
axes[0, 1].set_title("Spend by city (dots = outliers)")

sns.scatterplot(data=df, x="age", y="spend",
                hue="city", alpha=0.7, ax=axes[1, 0])
axes[1, 0].set_title("Age vs spend")

sns.heatmap(df.corr(numeric_only=True), annot=True,
            cmap="Blues", ax=axes[1, 1])
axes[1, 1].set_title("Correlations")

fig.tight_layout()
fig.savefig("overview.png", dpi=150, bbox_inches="tight")
```



Detect first

IQR bounds, before any plotting



Print the rows

never treat what you have not seen



2 x 2 grid

axes[0, 0] — Day 12 indexing



histplot

the shape of one column



boxplot

the IQR rule, drawn



hue=

a third variable, free



numeric_only=True

corr() needs it with text columns



tight_layout + bbox

nothing cropped, nothing overlapping

Four charts and an outlier check is a complete first look at a dataset. This is tomorrow's project in miniature.

Practice session — 30 minutes



Predict the output

IQR bounds, z-scores, plot defaults

8 questions



Spot & fix the bug

Z-score blind spot, barplot mean, labels

4 questions



Write the code

Detect, cap, hist, box, scatter with hue

5 tasks

Your notebook

[Day20_Practice_Questions.ipynb](#)

- Label every axis
- Print outlier rows before acting
- Swap charts with a neighbour
- Solutions shared after class

Every chart you make today must have both axes labelled and a title. An unlabelled chart is an unfinished chart.

TODAY IN TWELVE LINES

Recap

- 1 An outlier may be a typo, a sentinel, or the finding itself.
- 2 No test can tell you which. Print the rows and look.
- 3 IQR bounds: $Q1 - 1.5 * IQR$ and $Q3 + 1.5 * IQR$.
- 4 ⚠️ A z-score can MISS the outlier that distorts its own mean.
- 5 Keep, remove, cap with `clip()`, or transform — all have costs.
- 6 Anscombe: identical statistics, four completely different shapes.
- 7 Always `fig, ax = plt.subplots()` — never bare `plt.plot()`.
- 8 `line` · `bar` · `scatter` · `hist` cover most of what you need.
- 9 Label both axes and give every chart a title.
- 10 Seaborn takes `data=` and a column name, and gives you `hue=`.
- 11 ⚠️ `sns.barplot` shows the MEAN, with a confidence interval.
- 12 A boxplot IS the IQR rule, drawn — dots are the outliers.

Homework

Find the outliers in a dataset by IQR and by z-score and explain any disagreement · Rebuild the 2x2 overview grid for a dataset of your own · Take one chart and improve it until a classmate reads it correctly

Next class — Day 21

Mini project: a full EDA on a real dataset. Load, clean, explore, visualise and write up — everything from Days 15 to 20, end to end.